DevOps Explanation Request

Alright vicky 😈

DAY-30: DNS (the reason the internet works at all)

Let's go **clean** → **practical** → **no BS**, and I'll answer your last question properly.

Same flow 👇

👉 **way** → **problem** → **how DNS solves it** → **records** → **local DNS** → **frontend question**

☠️ Way (If DNS didn't exist)

Imagine this life:

- You type `142.250.183.206` instead of `google.com`
- IP changes → website breaks
- CDN changes IP → production outage
- Humans crying, computers chilling

So yeah… **DNS exists because humans suck at remembering numbers**.

💣 Problem

1. Machines talk using **IP addresses**
2. Humans prefer **names**
3. IPs change frequently:
    - CDN
    - Load balancers
    - Auto-scaling infra

We need a system that:

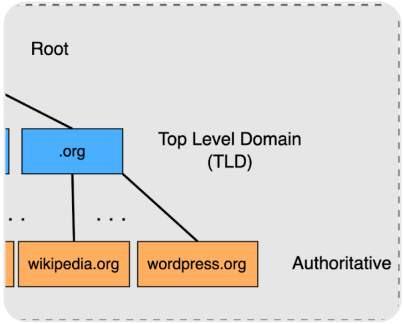
- Maps **name** → **IP**
- Updates dynamically
- Works globally

🛠️ How DNS Solves This

DNS = **Distributed phonebook of the internet**

```pgsql
Domain Name → IP Address
```

It works in **layers**, not one server.



https://www.researchgate.net/publication/342547942/figure/fig1/AS%3A907969327800320%4015934881657786/llustration-of-DNS-resolution-over-recursive-

---

Step 1 – DNS Hierarchy (Very Important)

Example:

`www.google.com`

Part	Meaning
`www`	Sub-domain
`google`	Apex / Root domain
`.com`	Top-Level Domain (TLD)

Root servers

- Know **where TLD servers are**
- They don't know IPs directly

Flow:

```mathematica
Root → .com → google.com → IP
```

Step 2 – DNS Record Types (The real power)

🔹 A Record (MOST USED)

```nginx
Domain → IPv4
```

Example:

```
google.com → 142.250.183.206
```

🔹 AAAA Record

```nginx
Domain → IPv6
```

Same as A record, modern internet version.

🔹 NS Record (Advanced)

```pgsql
Domain → Which DNS server is authoritative
```

Used when:

- Hosting your own DNS
- Delegating DNS responsibility

🔹 CNAME Record (VERY IMPORTANT)

Alias record.

Example:

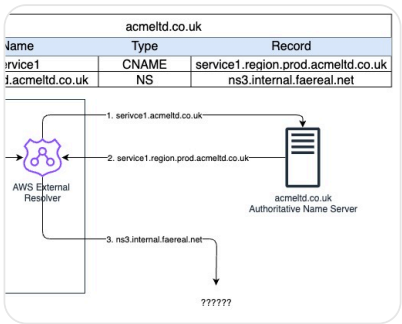
```
www.google.com → google.com
```

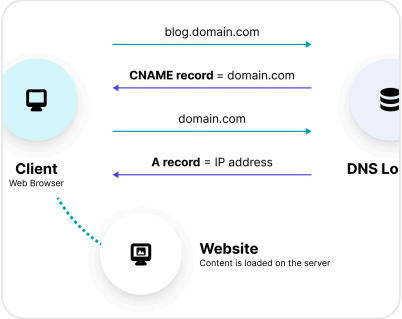
How resolution works:

1. Ask for `www.google.com`
2. DNS returns: **CNAME = google.com**
3. Then resolve `google.com`
4. Get IP
5. Use that IP

So effectively:

```ini
www.google.com === google.com
```





Step 3 – CDN / Load Balancer case (Why CNAME is king 👑)

Problem with A record + CDN

If you do:

```
myapp.com → 1.2.3.4
```

But CDN:

- Changes IP frequently
- Uses geo-routing
- Uses multiple IPs

💥 Your app breaks.

Correct approach (Production standard)

```objectivec
myapp.com → CNAME → myapp.cdn.cloudflare.net
```

Then:

```kotlin
Cloudflare DNS → actual IP
```

So:

- Your domain never changes
- CDN handles IP changes
- Zero downtime

👉 **This is why CNAME exists**

Step 4 – Local DNS (`/etc/hosts`)

For **testing only**.

File:

```bash
/etc/hosts
```

Example:

```text
127.0.0.1 app1.local
127.0.0.1 app2.local
127.0.0.1 app.local
```

What happens:

- OS checks `/etc/hosts` FIRST
- DNS servers are bypassed
- Only works on **your machine**

⚠️ No propagation
⚠️ No TLS (unless configured)
⚠️ Not scalable

Step 5 – The IMPORTANT Question 🔥

> **When testing frontend we use `localhost`,**
> **in production we use backend URL.**
> **Instead of localhost, can we use local DNS?**

✅ YES — and you SHOULD

This is actually **best practice**.

🔥 Why `localhost` is BAD for frontend dev

Problems:

- Hardcoded URL
- Environment mismatch
- CORS issues
- Painful production switch

✅ Correct way (Real-world best practice)

Development

Use local DNS:

```lua
api.dev.local
```

Add to `/etc/hosts`:

```text
127.0.0.1 api.dev.local
```

Frontend config:

```js
API_URL = http://api.dev.local
```

Production

DNS:

```
api.mycompany.com
```

Frontend config:

```js
API_URL = https://api.mycompany.com
```

Result 🎯

- Same **code**
- Same **logic**
- Only DNS changes

This is how professionals do it.

Step 6 – Why this matters even more in Kubernetes

Inside Kubernetes:

- Pods NEVER use `localhost` for services
- They use **DNS names**

Example:

```pgsql
backend.default.svc.cluster.local
```

So training your app to use DNS early:

👉 makes Kubernetes migration painless

Step 7 – Local DNS vs Real DNS (Never confuse)

Feature	`/etc/hosts`	Real DNS
Scope	Local machine	Global
Dynamic	×	✅
CDN support	×	✅
Production	×	✅
Kubernetes-style	×	✅

🧠 Final Mental Model (Lock it)

```sql
DNS = abstraction over IP
CNAME = abstraction over IP changes
localhost = shortcut (bad habit)
local DNS = production-like testing
```

Or simply:

> **If your app works with DNS names,**
> **it will work everywhere.**

If you want next 😉🔥

- DNS + TLS (why certs break on localhost)

- Why `.local` sometimes fails (mDNS gotcha)
- CoreDNS vs Internet DNS comparison
- StatefulSet DNS naming

Say the word.